PŘEKRÝVÁNÍ TĚLES
PHONGŮV OSVĚTLOVACÍ MODEL

Kurz: **Moderní počítačová grafika**

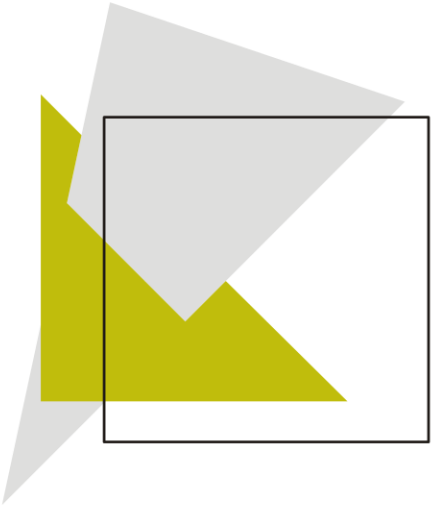
Lektor: Ing. Michal Švento

Náplň cvičení

1. Problém překrývání těles
   - Implementace paměti hloubky
2. Efekt mlhy
3. Phongův osvětlovací model
   - Materiálové vlastnosti
   - Normálové osvětlení
   - Nastavení světla, světelné zdroje
   - Změna vykreslování ploch
4. Programový výpočet normál

Problém překrývání těles

- při zobrazení více ploch
  - plochy se vykreslují v pořadí, v jakém byly definovány
- **paměť hloubky (z-buffer)**
  - výpočetní přístup spočívá v rozhodování na základě hloubkové informace jednotlivých fragmentů
  - provede se po aplikaci všech transformací při převodu souřadnic na 2D



∞	∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞	∞

+

5	5	5	5	5	5	5
5	5	5	5	5	5	
5	5	5	5	5		
5	5	5	5			
5	5	5				
5	5					
5						

=

5	5	5	5	5	5	5	∞
5	5	5	5	5	5	∞	∞
5	5	5	5	5	∞	∞	∞
5	5	5	5	∞	∞	∞	∞
5	5	5	∞	∞	∞	∞	∞
5	5	∞	∞	∞	∞	∞	∞
5	∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞	∞

5	5	5	5	5	5	5	∞
5	5	5	5	5	5	∞	∞
5	5	5	5	5	∞	∞	∞
5	5	5	5	∞	∞	∞	∞
5	5	5	∞	∞	∞	∞	∞
5	5	∞	∞	∞	∞	∞	∞
5	∞	∞	∞	∞	∞	∞	∞
∞	∞	∞	∞	∞	∞	∞	∞

+

7					
6	7				
5	6	7			
4	5	6	7		
3	4	5	6	7	
2	3	4	5	6	7

=

5	5	5	5	5	5	5	∞
5	5	5	5	5	5	∞	∞
5	5	5	5	5	∞	∞	∞
5	5	5	5	∞	∞	∞	∞
4	5	5	7	∞	∞	∞	∞
3	4	5	6	7	∞	∞	∞
2	3	4	5	6	7	∞	∞
∞	∞	∞	∞	∞	∞	∞	∞

Implementace paměti hloubky

- `glutInitDisplayMode( … | GLUT_DEPTH)`
- k zapnutí slouží `glEnable(GL_DEPTH_TEST)`, obdobně vypnutí
- mazání bufferu pomocí `glClear( … | GL_DEPTH_BUFFER_BIT)`
- nastavení testování hloubky
  - `glClearDepth(float depth)`
    - nastavuje maximální hloubku, kterou buffer maže
    - hodnoty v rozmezí 0 až 1
    - prakticky se nastavuje vždy 1
    - nastavuje se před voláním `glClear()`
  - `glDepthFunc(enum func)`
    - specifikuje funkci vyhodnocování fragmentů vůči aktuální hodnotě z-bufferu tak, aby kladný výsledek znamenal vykreslení
    - praktické nastavení je `GL_LESS`, existuje více módů

Implementace paměti hloubky

- `glClearDepth(1.0);` // hloubka mazani z-bufferu
- `glClearColor(0.5, 0.5, 0.5, 0.0);` // barva mazani z-bufferu
- `glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);` // mazani
- `glEnable(GL_DEPTH_TEST);` // zapnuti z-buffer testu
- `glDepthFunc(GL_LESS);` // nastaveni srovnavaci funkce

Úkol 1

1. Doplňte funkci `gluLookAt()` se správnými parametry, aby v kameře byl objekt viditelný.
2. Kamera v bodě [4, 12, 60].
3. Implementací paměti hloubky zamezte nesprávnému překrývání objektů.
4. Pomocí funkce `drawCube()` přidejte další objekty a testujte z-buffer

01_prekryvani.cpp

Efekt mlhy

- v dané vzdálenosti od přední projekční roviny začne plynulý přechod do jednotné barvy, které dosáhne v druhé dané vzdálenosti
- `glEnable(GL_FOG)`
- `glFogf(enum param, float value)`
  - `GL_FOG_START` – vzdálenost začátku mlhy
  - `GL_FOG_END` – vzdálenost „úplné“ mlhy
  - `GL_FOG_DENSITY` – pro potřeby exponenciálního přechodu
- `glFogfv(GL_FOG_COLOR, float value)`
  - nastavení barvy „úplné“ mlhy pomocí pole s hodnotami R G B A
- `glFogi(GL_FOG_MODE, enum mode)`
  - `GL_LINEAR`, `GL_EXP`, `GL_EXP2` – lineární nebo exponenciální přechod

Efekt mlhy

- `glHint(enum target, enum mode)`
  - specifikuje nastavení kvality různých parametrů vykreslování
- target:
  - `GL_FOG_HINT` – přesnost vykreslování mlhy
  - `GL_PERSPECTIVE_CORRECTION_HINT` – přesnost perspektivní korekce při zobrazování textur
  - `GL_LINE_SMOOTH_HINT` – přesnost vyhlazování čar
  - `GL_POINT_SMOOTH_HINT` – přesnost vyhlazování bodů
  - `GL_POLYGON_SMOOTH_HINT` – přesnost vyhlazování polygonů
- mode:
  - `GL_FASTEST` – nejrychlejší vykreslení s nejnižší kvalitou
  - `GL_NICEST` – nejpomalejší vykreslení s nejvyšší kvalitou
  - `GL_DONT_CARE` – výchozí nastavení dle prostředí a akcelerátoru

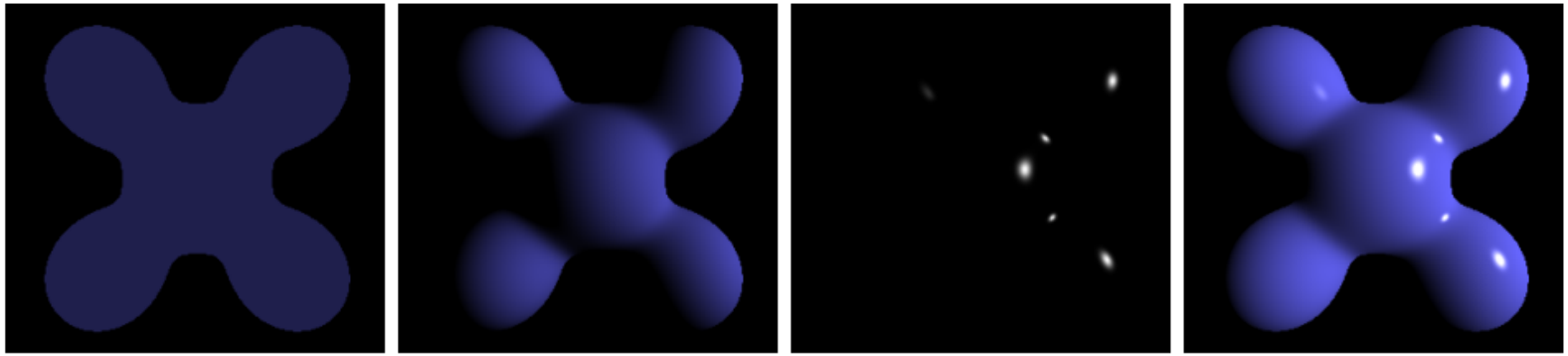
Úkol 2

1. Realizujte efekt mlhy v předchozím příkladu s perspektivní projekcí.
   - `near = 0.1`
   - `far = 300`
   - `fovy = 50`
2. Mlha:
   - lineární
   - start ve vzdálenosti 40, konec ve vzdálenosti 80
   - barva 50% šedá
   - maximální kvalita vykreslování

01_prekryvani.cpp

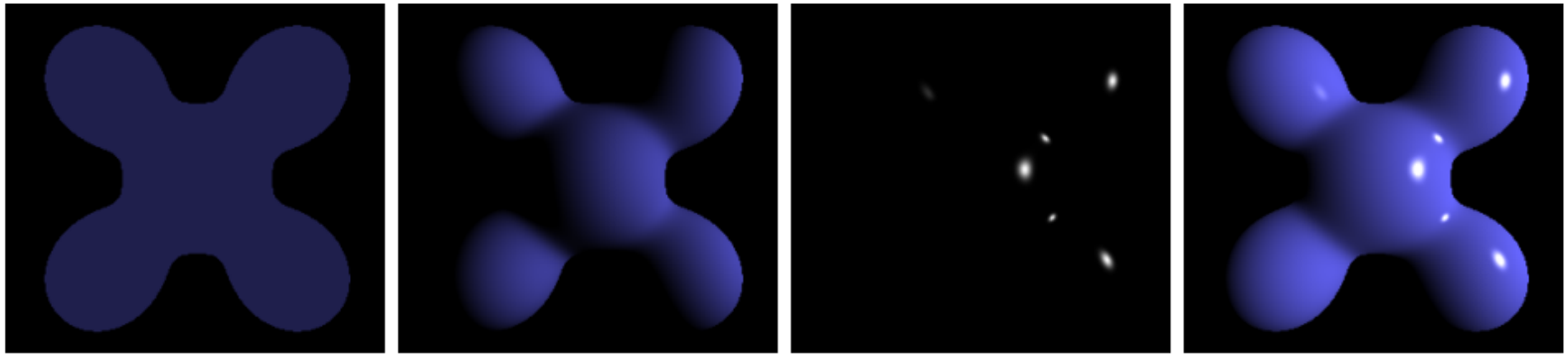
Phongův osvětlovací model

- relativně nízká hardwarová náročnost
- zapnutím osvětlení se přestane barva vykreslovaných fragmentů generovat z vlastnosti color jednotlivých vertexů
- použijí se materiálové vlastnosti



Materiálové vlastnosti

- ambientní složka
  - okolní světlo dopadající rovnoměrně na celé těleso
  - všesměrové konstantní osvětlení
- difúzní složka
  - světlo dopadající na těleso z konkrétního zdroje a rovnoměrně se rozptylující
- spekulární složka
  - lesk, neboli světlo, které dopadá na těleso a odráží se dle zákonu odrazu a lomu



Materiálové vlastnosti

- v OpenGL se vlastnosti nastavují vertexům
- `glMaterialf(enum face, GL_SHININESS, float value)`
  - `GL_FRONT`, `GL_BACK`, `GL_FRONT_AND_BACK`
  - specifikuje faktor odlesku
- `glMaterialfv(enum face, enum property, float value)`
  - `GL_FRONT`, `GL_BACK`, `GL_FRONT_AND_BACK`
  - `GL_AMBIENT` – ambientní barva
  - `GL_DIFFUSE` – difúzní barva
  - `GL_SPECULAR` – spekulární barva
  - `GL_EMISSION` – emitovaná barva
  - tyto 4 specifikované jako barva v prostoru R G B A

Normálové osvětlení

- pro aplikaci zákona o odrazu a lomu světla je nutné definovat normály ploch
- opět se definuje pro vertexy
- pokud není zapnutá automatická normalizace, musí být vektor jednotkový
  - `glEnable(GL_NORMALIZE)` nebo `glEnable(GL_RESCALE_NORMAL)`
- definice aktuální normály
  - platí pro všechny následující vertexy, dokud nedefinuji novou hodnotu
  - `glNormal(float x, y, z)`

Nastavení světla, světelné zdroje

1. zapnout osvětlení scény
   - `glEnable(GL_LIGHTING)`
2. nastavit typ stínování
   - `glShadeModel(enum model)`
     - `GL_FLAT` – celá plocha stínována stejně
     - `GL_SMOOTH` – aplikace Gouraudova stínování (dle nastavení jednotlivých vertexů, které plochu definují)
3. zapnout světelné zdroje
   - `glEnable(GL_LIGHTx)` – x je v rozmezí 0 až 7
4. nastavit vlastnosti světel

Nastavení světla, světelné zdroje

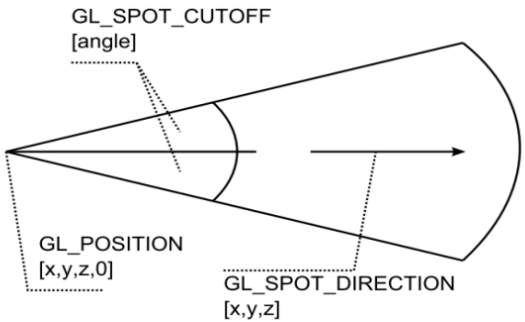
- pro jakýkoliv zdroj světla…
- `glLightfv(GL_LIGHTx, enum param, float value)`
  - `GL_AMBIENT` – intenzita světla v ambientní složce, opět jako R G B A
  - `GL_DIFFUSE` – intenzita světla v difúzní složce, opět jako R G B A
  - `GL_SPECULAR` – intenzita světla v spekulární složce, opět jako R G B A
  - `GL_POSITION` – pozice světla, viz další slajd

- následující pouze jestliže je zdroj reflektorový… (viz další slajd)
  - `GL_SPOT_DIRECTION`

- `glLightf(GL_LIGHTx, enum param, float value)`
  - `GL_SPOT_EXPONENT` – intenzita distribuce světla
  - `GL_SPOT_CUTOFF` – maximální úhel paprsků

Nastavení světla, světelné zdroje

- typy zdrojů světla
  - bodový
    - definován v jednom bodě, vyzařuje do všech stran stejně
    - parametr `GL_POSITION` se nastavuje jako (x, y, z, 1)
  - směrový
    - zdroj nemá výchozí pozici, paprsky jsou rovnoběžné
    - parametr `GL_POSITION` se nastavuje jako (x, y, z, 0) a specifikuje vektor směru vyzařování
  - reflektorový
    - bodový zdroj, který však vyzařuje omezeně
    - `GL_POSITION`
    - `GL_SPOT_CUTOFF`
    - `GL_SPOT_DIRECTION`
    - `GL_SPOT_EXPONENT`



Změna vykreslování ploch

- syntaxe funkcí pro změnu vykreslování ploch a nastavení stínování:
  - `glFrontFace(enum mode)` – mód vykreslování „předních“ stěn
    - `GL_CW` – clockwise, po směru hodinových ručiček
    - `GL_CCW` – counter clockwise
  - `glPolygonMode(enum face, enum mode)` – mód vykreslování dané stěny
    - `GL_FRONT, GL_BACK, GL_FRONT_AND_BACK` – výběr stěn
    - `GL_POINT, GL_LINE, GL_FILL` – způsob vykreslení
- možná úspora výpočetních prostředků pomocí ignorování některých stěn:
  - `glCullFace(enum face)` – ořez stěn
    - `glEnable(GL_CULL_FACE)` – nutné zapnutí, aby ořez fungoval

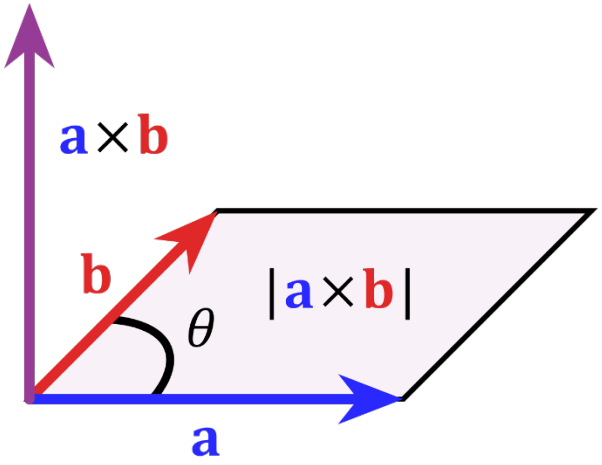
Úkol 3

02_osvetleni.cpp

1. V připraveném zadání doplňte do bloku `onDisplay()` definici a zapnutí světel a zobrazte krychli o straně 20, která má střed v počátku souřadnic. Krychli přidejte korektní normály vzhledem ke všem šesti plochám (1 tatáž normála bude společná pro 4 vertexy, které definují jednu stěnu).
2. Využijte předdefinovaných polí a hodnot `lightPosition` atd. ve zdrojovém kódu. Vlastnosti světla aplikujte např. na světlo číslo 0.
3. Při definici krychle si dávejte pozor na správný směr definice vertexů. Je zapnuto zobrazování pouze strany stěn `FRONT`, které se definují ve směru hodinových ručiček. Potlačení zobrazení zadních stěn ověřte odebráním jedné stěny krychle a „podíváním se“ dovnitř krychle.
4. Byl zapnut mód stínování `GL_SMOOTH` (Gouraudovo stínování, funkce `glShadeModel`), což znamená, že změníme-li normálu jednoho vertexu kompletního primitiva, dojde k lokální změně osvětlení v tomto vertexu. Vyzkoušejte. Pro změněnou normálu jednoho vertexu sledujte změny stínování, když přepnete do módu `GL_FLAT`.

Programový výpočet normál

- normálu k ploše lze v 3D prostoru vypočítat, pokud máme k dispozici 3 body z této plochy `vert1`, `vert2` a `vert3`
- z těchto tří bodů můžeme vypočítat dva vektory `v12` (v obrázku řekněme **a**) a `v13` (v obrázku řekněme **b**) a jejich vektorový součine (v obrázku **fialový vektor**)
- tento vektor je na oba předchozí kolmý (tudíž je to normála)
- abychom zajistili jednotkovou velikost, podělíme celý vektor jeho velikostí



```
void vypoctiNormalu(float vert1[], float vert2[], float vert3[], float *nx, float *ny, float *nz)
{
	// vektor od vert1 do vert2
	float v12x, v12y, v12z;

	// vektor od vert1 do vert3
	float v13x, v13y, v13z;
	float invnorm;

	// výpočet vektorů
	v12x = vert2[0] - vert1[0];
	v12y = vert2[1] - vert1[1];
	v12z = vert2[2] - vert1[2];
	v13x = vert3[0] - vert1[0];
	v13y = vert3[1] - vert1[1];
	v13z = vert3[2] - vert1[2];

	// vektorový součin v12 x v13
	*nx = v12y * v13z - v12z * v13y;
	*ny = v12z * v13x - v12x * v13z;
	*nz = v12x * v13y - v12y * v13x;

	// 1 / norma vektorového součinu
	invnorm = 1.0f / sqrt((*nx)*(*nx) + (*ny)*(*ny) + (*nz)*(*nz));

	// normování
	*nx *= invnorm;
	*ny *= invnorm;
	*nz *= invnorm;
}
```

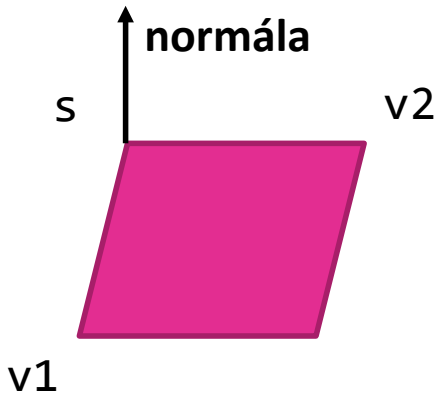
Programový výpočet normál

- komentáře ke kódu na předchozím snímku
  - vstupy jsou 3 pole o (alespoň) třech souřadnicích typu float, tedy 3 vertexy
    - `float vert1[], float vert2[], float vert3[]`
  - pomocí těchto vertexů spočítáme 3 složky 2 vektorů v12 a v13 (celkem tedy 6 hodnot)
  - `float *nx, float *ny, float *nz` chápeme jako výstupy funkce, jedná se o ukazatele na tři souřadnice výsledné normály (proto hvězdičky)
  - výpočet vektorového součinu viz matematika

Úkol 4

03_normaly.cpp

1. K jednotlivým polygonům útvaru doplňte definici správných normál.
   - souřadnice normál ukládejte do připraveného vektoru `float n[3]`
   - použijete-li funkce `glNormal3fv` a `glVertex3fv`, pak jim předejte celý vektor normály, resp. vertexu jako jediný argument
     - stejně tak to ale můžete vyřešit po jednotlivých složkách, pomocí `glNormal3f` a `glVertex3f`

Nápověda:

- stěny definujeme po směru hodinových ručiček
- musíme pohlídat pořadí vertexů tak, aby normála směřovala směrem ven z tělesa (pravidlo pravé ruky)